

0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0
0	1	1	1	1	0	0	0
0	0	1	1	1	1	0	0
0	0	0	1	1	1	1	1
0	1	1	1	1	0	0	0
0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0
0	1	1	1	1	1	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	1	1	1	1	1	1	0
0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	1	1	1	0	0
0	1	1	1	1	1	1	0
0	1	0	0	0	0	1	0
0	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0

apply the distance transform to approximate distances from the region containing 1's to all other pixels in the image, using the masks:

1

1 0 1

1

1 1 1

1 0 1

1 1 1

4 3 4

3 0 3

4 3 4

11 11

11 7 5 7 11

5 0 5

11 7 5 7 11

11 11

(i)

(ii)

(iii)

(iv)

- and applying any necessary scaling at the end.
- 28. Apply the distance transform and use it to find the skeleton in the images `circles2.png` and `nicework.png`.
 - 29. Compare the result of the previous question with the results given by the Zhang-Suen and Guo-Hall methods. Which method produces the most visually appealing results? Which method seems fastest?

12 Shapes and Boundaries

12.1 Introduction

In this chapter we shall investigate tools for examining image shapes. Some tools have already been discussed in Chapter 10; now we look at specific methods for examining shapes of objects. Questions we might ask about shapes include:

- How do we tell if two objects have the same shape?
- How can we classify shape?
- How can we describe the shape of an object?

Formal means of describing shapes are called *shape descriptors*. Shape descriptors may include size, symmetry, and length of perimeter. A precise definition of the exact shape in some efficient manner is a *shape representation*.

In this chapter we shall be concerned with the *boundary* of objects. A boundary differs from an edge in that whereas an edge is an example of a local property of an image, a boundary is a global property.

12.2 Chain Codes and Shape Numbers

The idea of a chain code is quite straightforward; we walk around the boundary of an object, taking note of the directions we take. The resulting list of direction is the chain code.

We need to consider two types of boundaries: 4-connected and 8-connected (see Chapter 11. If the boundary is 4-connected, there are four possible directions in which to walk; if the boundary is 8-connected, there are eight possible directions. These are shown in Figure 12.1.

To see how they work, consider the object and its boundary shown in Figure 12.2.

Suppose we walk along the boundary in a clockwise direction starting at the left-most point in the bottom row, and list the directions as we go. This is shown in Figure 12.3.

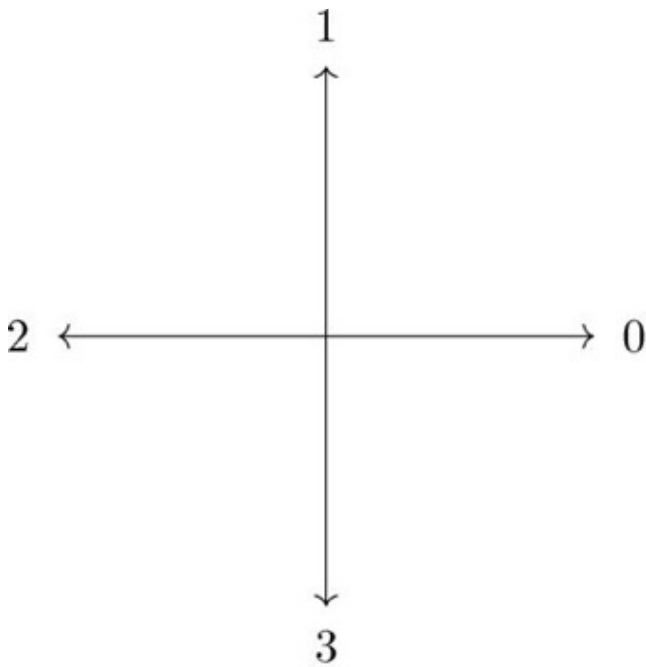
We can thus read off the chain code as:

3 3 3 2 3 3 0 0 0 1 0 1 1 1 2 1 2 2

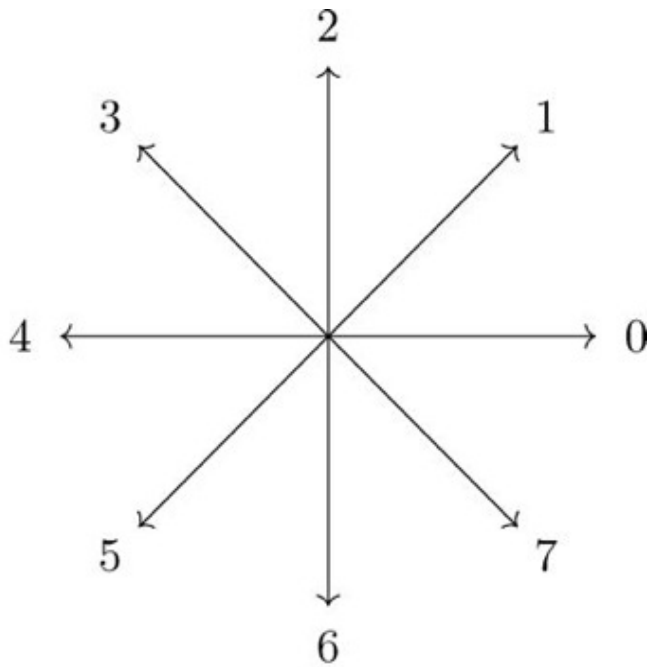
If we treat the object in Figure 12.2 as being 8-connected, then its boundary and chain code are generated as in the right-hand diagram in Figure 12.3. In this case, the resulting chain code is:

6 6 5 6 6 0 0 0 1 5 5 5 3 4 4

Figure 12.1: Directions for chain codes



Directions for 4-connectedness



Directions for 8-connectedness

2. Traverse the 3×3 neighborhood of the current pixel in an anticlockwise direction, beginning the search at the pixel in direction $\mathbf{dir} + 3 \pmod{4}$. This simply sets the current direction to the first direction anticlockwise from $\mathbf{dir}$:

$\mathbf{dir}$	0	1	2	3
$\mathbf{dir} + 3 \pmod{4}$	3	0	1	2

 The first foreground pixel will be the new boundary element. Update $\mathbf{dir}$.
3. Stop when the current boundary element P_n is equal to the second element P_1 and the previous boundary pixel P_{n-1} is equal to the first boundary element P_0 .

Suppose we have a binary image `im` consisting of a single object. We can find the top left pixel with the following MATLAB commands:

```
>> [x,y] = find(im==1);
>> x = min(x)
>> imx = im(x,:);
>> imy = min(imx)
```

MATLAB/Octave

or in Python with

```
In : x,y = np.where(im==1)
In : mx = x.min()
In : my = y[np.where(x==mx)].min()
```

Python

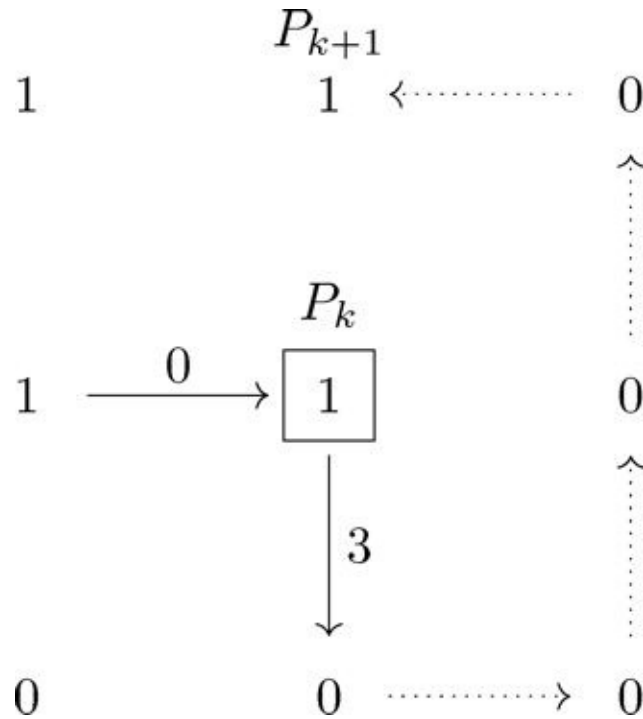
pixels. The second command finds the minimum of the first coordinates. Thus, `mx` is the top row of the object. The third command isolates this top row, and the final command finds the left-most column in it.

Given the indices `x` and `y` of the current pixel, and the value `dir`, how do we implement Step 2? We give a simple example, shown in Figure 12.4. Suppose that in this particular example, the current value of `dir` is 0 so that

$$\mathbf{dir} + 3 \pmod{4} = 3.$$

The dotted arrows indicate the direction of traversing (starting from the pixel in direction 3 from P_k) until we reach a new foreground pixel. This pixel is then P_{k+1} .

Figure 12.4: Traversing a neighborhood



We start by noting the row and column increments to the four-neighbors of a pixel:

$$\begin{array}{ccc} -1 & 0 & \\ 0 & -1 & 0 \quad 0 \quad 0 \quad 1 \\ 1 & 0 & \end{array}$$

Since we are looking at four-connected boundaries, these are the only neighbors we need. We can put these into a matrix, with the first row corresponding to the increments in direction 0:

$$n = \begin{bmatrix} 0 & 1 \\ -1 & 0 \\ 0 & -1 \\ 1 & 0 \end{bmatrix}$$

This means that the indices in the j -th row correspond to the increments in indices in direction $j - 1$. Thus, given a direction `dir`, we can enter:

```
>> newdir = mod(dir+3,4);
>> for i=0:3,
>     j=mod(newdir+i,4)+1;
>     tt(i+1) = im(x+n(j,1),y+n(j,2)),
> end
```

MATLAB/Octave

or

```
In : newdir = mod(dir+3,4)
In : for i in range(4):
...:     j = mod(newdir+i,4)
...:     tt[i] = im[x+n[j,0],y+n[j,1]]
```

Python

and this will traverse the neighborhood of our image `im` at position `(x, y)` starting from the correct direction. Notice that for MATLAB and Octave we need to set

```
>> j = mod(newdir+i,4)+1;
```

MATLAB/Octave

The extra “+1” takes account of the fact that the modulus function returns values 0, 1, 2, 3, but the rows of `n` are 1, 2, 3, 4. The vector `tt` contains the values of the neighborhood as it is traversed.

The first non-zero value is easily found with

```
>> d = min(find(tt==1));
```

MATLAB/Octave

or with

```
In : d = min(flatnonzero(tt==uint8(1)))
```

Python

and now we can update `dir`, and the position of the current pixel:

```
>> dir = mod(newdir+d-1,4);
>> x = x+n(dir+1,1);
>> y = y+n(dir+1,2);
```

MATLAB/Octave

or

```
In : dir = mod(newdir+d,4)
In : x = x+n[dir,0]
In : y = y+n[dir,1]
```

Python

This most recent value of `dir` is placed into a vector, and this is the vector which will be the final chain code.

Functions for implementing the full chaincode are given at the end of the chapter.

These programs can be tested with the shape in Figure 12.2 (surrounding the image with zeros first):

```
>> test = zeros(8,7); test(2:7,3:5) = 1; test(5:7,2) = 1; test(3:6,6) = 1;
>> [cc, bdy] = shape(test, 4);
>> uint8(cc)
ans =

 3  3  3  2  3  3  0  0  0  1  0  1  1  1  2  1  2  2
```

MATLAB/Octave

and comparing this with the code given earlier, the function has indeed returned the correct chain code.

We can easily modify the program to perform chain codes for 8-connected boundaries. The above algorithm must be slightly changed; again, see Sonka et al. [49].

1. Start by finding the pixel in the object that has the left most value in the topmost row; call this pixel P_0 . Define a variable `dir` (for direction), and set it equal to 7. (Since P_0 is the top left pixel in the object, the direction to the next pixel must be 7.)

2. Traverse the 3×3 neighborhood of the current pixel in an anticlockwise direction,

beginning the search at the pixel in direction $\begin{matrix} \text{dir} + 7 \pmod{8} & \text{if } \text{dir} \text{ is even} \\ \text{dir} + 6 \pmod{8} & \text{if } \text{dir} \text{ is odd} \end{matrix}$ This simply sets the current direction to the first direction anticlockwise from `dir`:

<code>dir</code>		0	1	2	3	4	5	6	7
<code>dir + 7 (mod 8)</code>		7	0	1	2	3	4	5	6
<code>dir + 6 (mod 8)</code>		6	7	0	1	2	3	4	5

The first foreground pixel will be the new boundary element. Update `dir`.

3. Stop when the current boundary element P_n is equal to the second element P_1 and the previous boundary pixel P_{n-1} is equal to the first boundary element P_0 .

Note that the choosing of the starting direction in Step 2 can be implemented by

```
newdir = mod(dir+7-mod(dir,2),8);
```

MATLAB/Octave

or

```
newdir = (dir+7-dir%2)%8
```

Python

which produces `dir + 7` if `dir` is even, and `dir + 6` if `dir` is odd. As well as this, we need to take into account *all* possible directions from a pixel; the index increments are:

$$\begin{array}{ccc}
 -1 & -1 & -1 & 0 & -1 & 1 \\
 0 & -1 & 0 & 0 & 0 & 1 \\
 1 & -1 & 1 & 0 & 1 & 1
 \end{array}$$

These will all be placed in the array

$$n = \begin{bmatrix} 0 & 1 \\ -1 & 1 \\ -1 & 0 \\ -1 & -1 \\ 0 & -1 \\ 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

where as before, direction 0 corresponds to the top row of n . This can be tested on the test image `test`:

```
>> uint8(shape(test,8))
ans =
```

```
6 6 5 6 6 0 0 0 1 2 2 2 3 4 4
```

MATLAB/Octave

and the answer is indeed the code obtained earlier by following the arrows around the object.

Programs are given at the end of the chapter.

Normalization of Chain Codes

There are two problems with the definition of the chain code as given in previous sections:

1. The chain code is dependent on the starting pixel
2. The chain code is dependent on the orientation of the object

First look at Problem (1). The idea is to “normalize” the chain code as follows: imaging the code to be written around the edge of a circle. We choose as our starting place the position for which the code, when read off, will be the lowest possible integer. The result is the *normalized chain code* for our object.

For example, suppose we have an object consisting of a 3×3 square:


```
>> a=zeros(5,5);a(2:4,2:4)=1
```

```
0  0  0  0  0
0  1  1  1  0
0  1  1  1  0
0  1  1  1  0
0  0  0  0  0
```

```
>> shape(a,4)
```

```
ans =
```

```
3  3  0  0  1  1  2  2
```

MATLAB/Octave

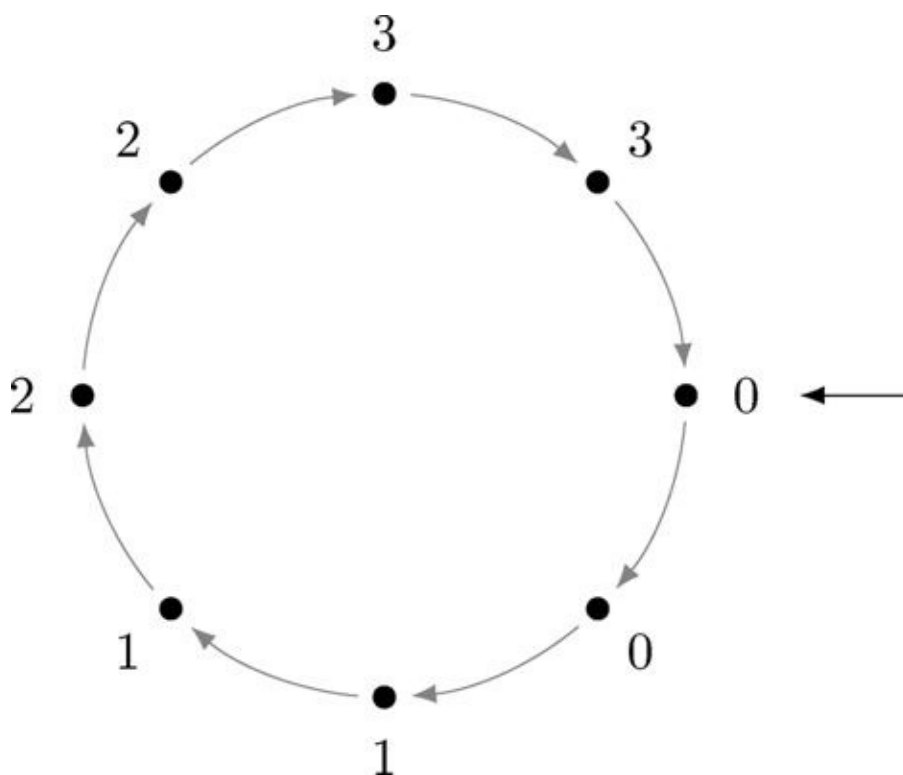
Now let's put these codes around a circle as shown in Figure 12.5.

The arrow indicates where we should start reading off the code to obtain the lowest integer; in this case it is

0 0 1 1 2 2 3 3

But we can do this easily in MATLAB or Octave by using the `circshift` function to create every possible shift. For our chain code above, we can do this:

Figure 12.5: A chain code written cyclically



```
>> N = length(c);
>> z = zeros(N);
>> for i = 1:N, z(i,:) = circshift(c',i)';end
>> z
ans =
```

2	3	3	0	0	1	1	2
2	2	3	3	0	0	1	1
1	2	2	3	3	0	0	1
1	1	2	2	3	3	0	0
0	1	1	2	2	3	3	0
0	0	1	1	2	2	3	3
3	0	0	1	1	2	2	3
3	3	0	0	1	1	2	2

MATLAB/Octave

To find the row that contains the least integer, we use the handy `sortrows` function, which sorts the rows “lexicographically”: first on the first element, then on the second element, and so on:

```
>> zs = sortrows(z)
zs =
```

0	0	1	1	2	2	3	3
0	1	1	2	2	3	3	0
1	1	2	2	3	3	0	0
1	2	2	3	3	0	0	1
2	2	3	3	0	0	1	1
2	3	3	0	0	1	1	2
3	0	0	1	1	2	2	3
3	3	0	0	1	1	2	2

MATLAB/Octave

Now the normalized chain code is just the first row:

```
>> zs(1,:)
ans =
```

0	0	1	1	2	2	3	3
---	---	---	---	---	---	---	---

MATLAB/Octave

In Python this can also easily be done; first create the test object and obtain its chain code:

```
In : a = zeros((5,5))
In : a[1:4,1:4]=1
In : c = shape(a,4)
```

Python

Now create an array whose rows contain the cyclic shifts; these can be obtained using the `roll` method of `numpy`:

```
In : N = len(c)
In : z = zeros((N,N))
In : for i in range(N):
...:     z[i,:] = np.roll(c,i)
...:
```

Python

The array `z` can now be sorted in place along whichever axis we choose. For sorting the rows, the axis number is 1:

```
In : z.sort(axis=1)
In : print(z[0,:])
[ 0.  0.  1.  1.  2.  2.  3.  3.]
```

Python

Let's try this with a slightly larger example: the `test` object from Figure 12.2:

```
>> c = shape(test,4);
>> N = length(c);
>> z = zeros(N)
>> for i = 1:N, z(i,:) = circshift(c',i)';end
>> zs = sortrows(z);
>> zs(1,:)
```

ans =

```
0 0 0 1 0 1 1 1 2 1 2 2 3 3 3 2 3 3
```

MATLAB/Octave

This can be easily turned into a function. With Python we just repeat the commands above, starting with the new chain code array.

Shape Numbers

We now consider the second problem from above: defining a chain code that is independent of orientation of the object. For example, consider a simple “L” shape:

```
>> L = zeros(7,6); L(2:6,2:3) = 1; L(5:6,4:5) = 1
```

```
L =
```

```

0 0 0 0 0 0
0 1 1 0 0 0
0 1 1 0 0 0
0 1 1 0 0 0
0 1 1 1 1 0
0 1 1 1 1 0
0 0 0 0 0 0

```

MATLAB/Octave

The normalized chain code can be found by the commands above to be

```
0 0 0 1 2 2 1 1 1 2 3 3 3 3
```

Now suppose we rotate the shape so that it has a different orientation (we use the `rot90` function which rotates a matrix by 90 degrees):

```
>> L2 = rot90(L)
```

```
L2 =
```

```

0 0 0 0 0 0 0
0 0 0 0 1 1 0
0 0 0 0 1 1 0
0 1 1 1 1 1 0
0 1 1 1 1 1 0
0 0 0 0 0 0 0

```

MATLAB/Octave

The normalized chain code for this new orientation can be computed to be: 0000111233 22 23

```
0 0 0 0 1 1 1 2 3 3 2 2 2 3
```

Even when normalized, the chain codes are different.

To overcome this, take the *differences* of the chain code: for each two consecutive elements c_i and c_{i+1} , their difference is defined as

$$c_{i+1} - c_i \pmod{4}.$$

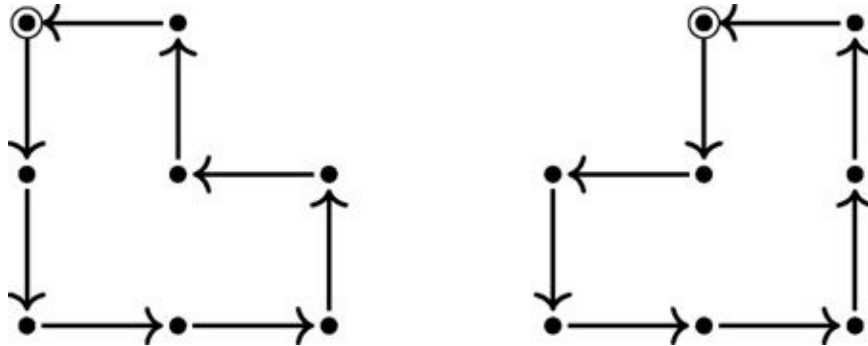
(If we were dealing with 8-connected boundaries, then we would take differences mod 8). For an example, take the simple L shape and its rotation shown in Figure 12.6. The chain code for the first shape can be read off easily starting from the point shown; it is

```
3 3 0 0 1 2 1 2.
```

To apply differences, subtract this list from a cyclic shift of itself:

$$\begin{array}{r}
 3 \ 3 \ 0 \ 0 \ 1 \ 2 \ 1 \ 2 \\
 - \ 2 \ 3 \ 3 \ 0 \ 0 \ 1 \ 2 \ 1 \\
 \hline
 = \ 1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 3 \ 1 \pmod{4}
 \end{array}$$

Figure 12.6: A simple L-shape



The normalized version of these differences is the *shape number* for this L shape and can be found to be

$$0 \ 1 \ 0 \ 1 \ 1 \ 3 \ 1 \ 1.$$

The chain code for the rotated L can be found to be

$$3 \ 2 \ 3 \ 0 \ 0 \ 1 \ 1 \ 2.$$

Even when normalized the result is not the same as the chain code for this shape in its original orientation. But if the differences are obtained:

$$\begin{array}{r}
 3 \ 2 \ 3 \ 0 \ 0 \ 1 \ 1 \ 2 \\
 - \ 2 \ 3 \ 2 \ 3 \ 0 \ 0 \ 1 \ 1 \\
 \hline
 1 \ 3 \ 1 \ 1 \ 0 \ 1 \ 0 \ 1
 \end{array}$$

then the normalized third row is

$$0 \ 1 \ 0 \ 1 \ 1 \ 3 \ 1 \ 1.$$

which is exactly the same code obtained above.

This can be done easily in MATLAB or Octave: given a chain code c , simply normalize the difference with a cyclic shift:

```
>> c = shape(a)
c =

    3  3  0  0  1  1  2  2
>> d = mod(c - circshift(c',1)',4)
d =

    1  0  0  0  1  0  1  0
>> chain_norm(d)
ans =

    0  0  0  1  0  1  0  1
```

MATLAB/Octave

Let's try this with our L shape and its rotation.

```
>> c = shape(L,4);
>> d = mod(c - circshift(c',1)',4);
>> chain_norm(d)
ans =

    0  0  0  1  0  0  1  1  0  3  0  0  1  1

>> c2 = shape(L2,4);
>> d2 = mod(c2 - circshift(c2',1)',4);
>> chain_norm(d2)
ans =

    0  0  0  1  0  0  1  1  0  3  0  0  1  1
```

MATLAB/Octave

and both results are exactly the same. With Python, the following commands obtain the same outcome:

```
In : c = shape(a,4)
In : d = (c-np.roll(c,1))%4
In : print(d)
[1 0 1 0 1 0 1 0]

In :
```

Python

12.3 Fourier Descriptors

The idea is this: suppose we walk around an object, but instead of writing down the directions, we write down the boundary coordinates. The final list of (x, y) coordinates can be turned into a list of complex numbers $z = x + yi$. The Fourier transform of this list of numbers is a *Fourier descriptor* of the object.

The beauty of a Fourier descriptor is that often only a few low frequency terms around the DC coefficient are enough to distinguish objects, or to classify them.

The chain code functions at the end of the chapter also produce the list of boundary pixels, so that, for example, with the L shape from the previous section:

```
>> [cc, bdy] = shape(L);  
>> bdy'  
ans =  
  
    3    4    5    6    6    6    6    5    5    5    4    3    2    2  
    2    2    2    2    3    4    5    5    4    3    3    3    3    2
```

MATLAB/Octave

Turning these into complex numbers is easy:

```
>> c = complex(bdy(:,1), bdy(:,2)); c'  
ans =  
  
Columns 1 through 8:  
  
    3 - 2i    4 - 2i    5 - 2i    6 - 2i    6 - 3i    6 - 4i    6 - 5i    5 - 5i  
  
Columns 9 through 14:  
  
    5 - 4i    5 - 3i    4 - 3i    3 - 3i    2 - 3i    2 - 2i
```

MATLAB/Octave

The same array can be obtained with Python:

```
In : cc, bdy = shape(L)  
In : c = bdy[:,0] + bdy[:,1]*1j
```

Python

These can be plotted:

```
>> plot(c, '-.', "markersize", 10), axis([1, 7, 1, 6]), axis equal
```

MATLAB/Octave

or with

```
In : import matplotlib.pyplot as plt
In : plt.plot(c.real,c.imag,'.-b',markersize=10)
In : plt.axis([0,6,0,5])
In : plt.show()
```

Python

and the result is shown in Figure 12.7. Supposing we take the Fourier transform, and from it extract only a few terms:

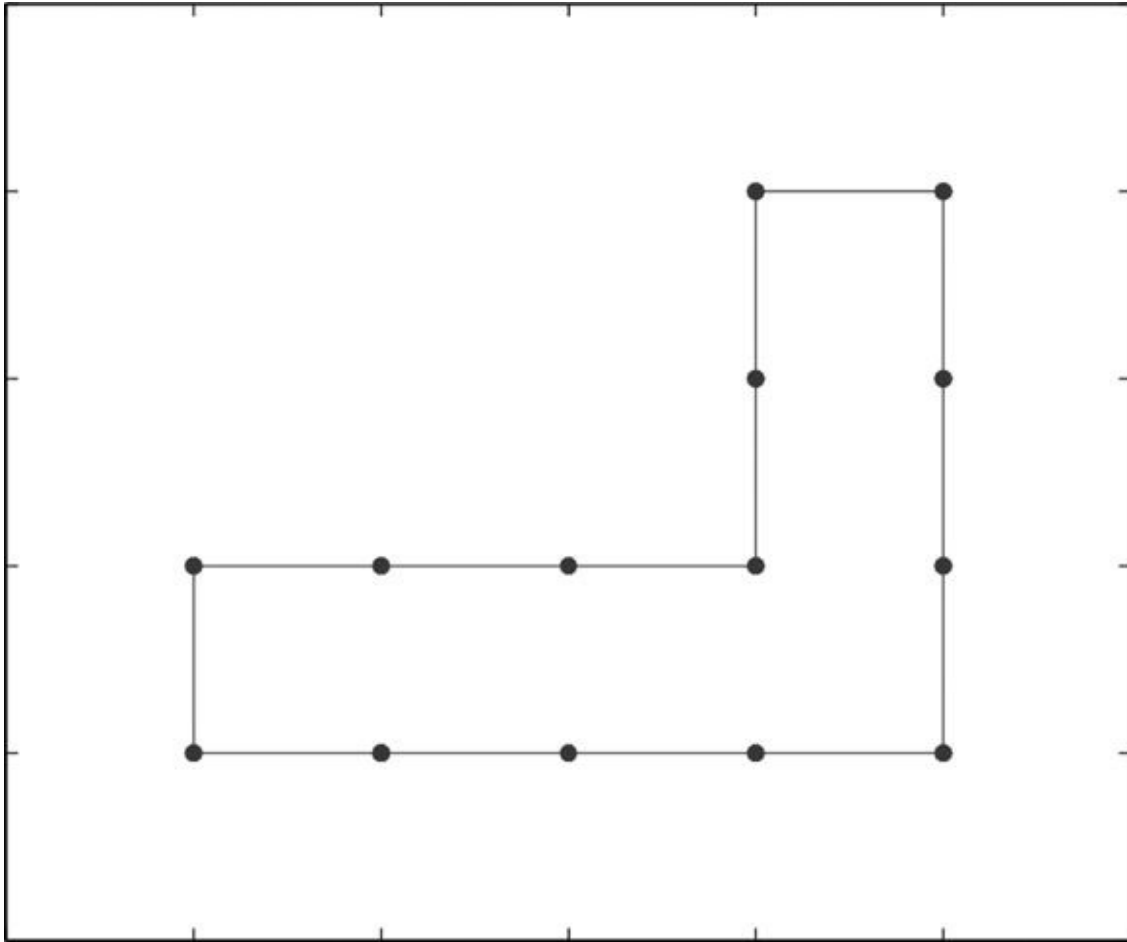
```
>> cf = fft(c)

cf =

62.00000 + 43.00000i
-9.76237 - 18.27796i
-5.69003 - 0.64111i
 2.87152 - 2.43834i
 0.52068 + 0.82866i
 1.45451 - 0.95368i
-0.16670 + 0.47640i
 0.00000 + 1.00000i
-0.47640 + 0.16670i
 0.75862 + 0.27672i
-0.82866 - 0.52068i
-0.64200 - 0.93602i
 0.64111 + 5.69003i
-8.68028 + 0.32927i
```

MATLAB/Octave

Figure 12.7: Boundary pixels



Note that as this transform is not shifted, the first term will be the DC coefficient, and low frequency terms occur after the DC coefficient, and at the end. What we can do is pick a value k , and keep k terms after the DC coefficient, and at the end:

```
>> k = 2  
>> cf2 = cf;  
>> N = len(cf);  
>> cf2(k+2:N-k) = 0;
```

MATLAB/Octave

At this stage, 9 of the original 14 values have been removed. They can be plotted with lines and points, and to ensure the lines meet up the first values will be repeated as the last:

```
>> bdy2 = ifft(cf2);  
>> bdy2 = [bdy2;bdy2(1,:)];  
>> plot(bdy2,'.-','markersize',10),axis([1,7,1,6]),axis equal
```

MATLAB/Octave

The result is shown in Figure 12.8(a).

Images (b) and (c) show the result by keeping a few more pairs of transform values. However even with $k = 2$ there is enough information to gauge the size and rough shape of the object; and with $k = 4$ (five

transform values removed) the result is very close.

Recall that in an unshifted transform, the DC coefficient is at the start, and the high frequency values are toward the middle, as shown in Figure 12.9.

In this figure, the arrows give the direction of increasing frequency; the elements labeled X and Y are those of highest frequency. Around those central values, and excluding the DC coefficient, all other elements can be paired together with equal frequencies, as shown.

Figure 12.8: Fourier descriptors of an L shape

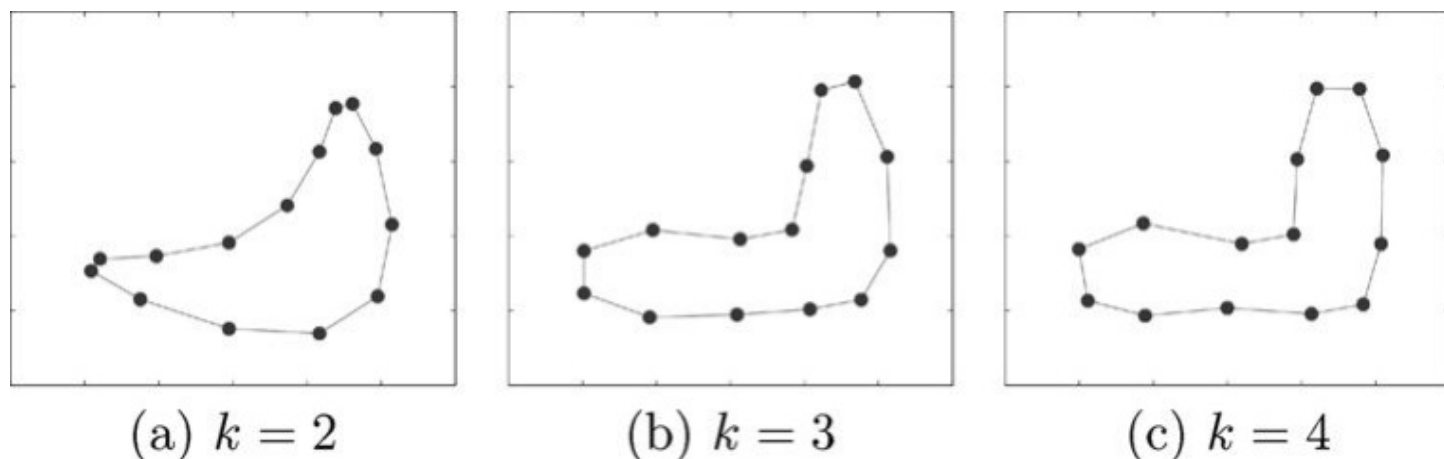
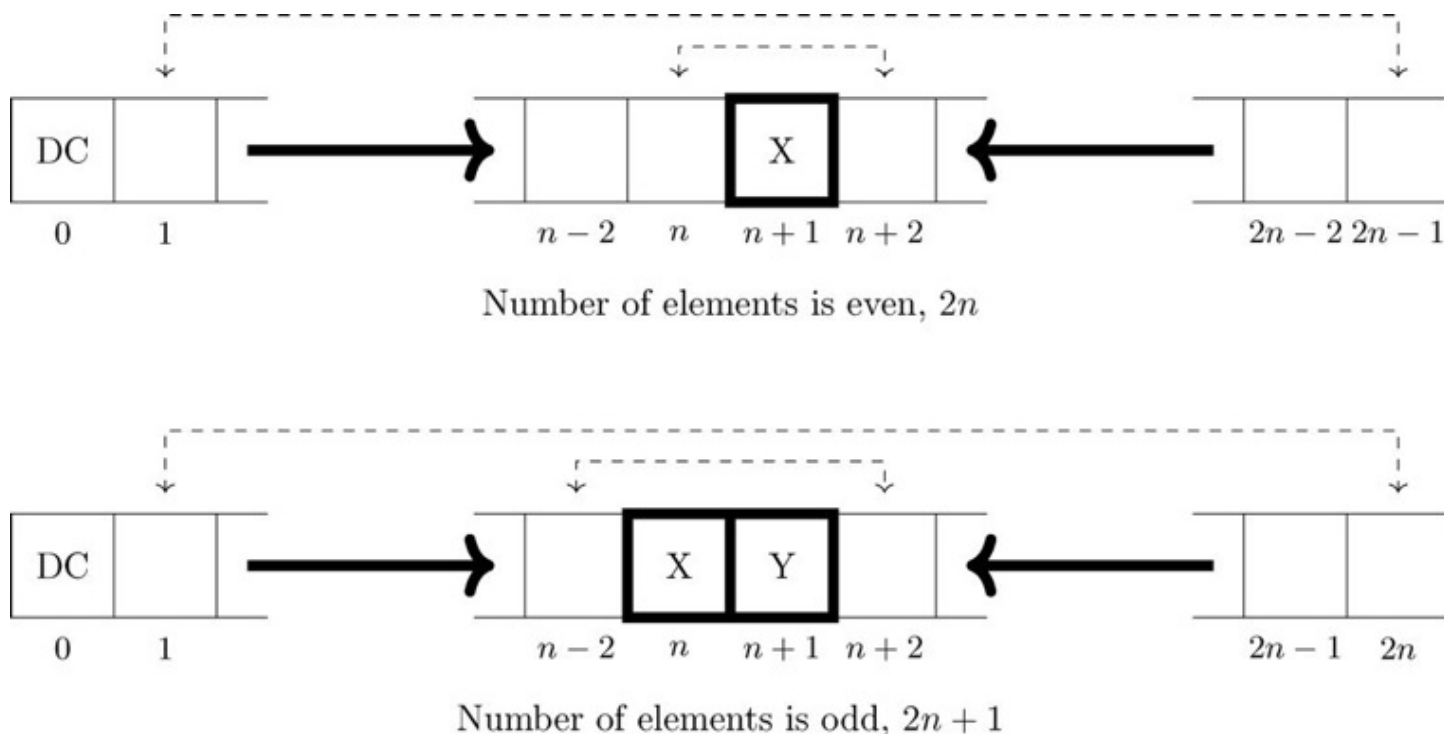


Figure 12.9: Unshifted one-dimensional DFT



This means that when we exclude elements from the transform, they must be excluded in pairs. In other words, given a transform with N elements, all but $2k + 1$ elements can be excluded as shown in Figure 12.10.

Figure 12.10: Excluding elements from the DFT

DC	k elements	$N - 2k - 1$ zeros	k elements
----	--------------	--------------------	--------------

In order to approximate a shape, given $2k + 1$ non-zero elements of a transform, there are several possibilities:

1. Apply the inverse transform to the complete N length vector (including the zeros), and plot the result.
2. Simply invert the smaller vector of $2k + 1$ elements, and plot those values joined by lines.
3. Place any number of zeros between the first $k + 1$ and last k values before inversion.

Consider a large shape with many boundary pixels:

```
>> z = zeros(256);  
>> z(32:224,32:96) = 1; z(32:96,96:224) = 1; z(192:224,96:160) = 1;  
>> [cc, bdy] = shape(z,4)
```

MATLAB/Octave

There are 896 pixels in the boundary. Now to transform them and delete most of them, first in MATLAB or Octave:

```
>> c = complex(bdy(:,1), bdy(:,2));  
>> N = length(c)  
N = 896  
  
>> cf = fft(c);  
>> k = 12;  
>> cf2 = cf; cf2(k+2:N-k) = 0;  
>> bdy2 = ifft(cf2);  
>> cf3 = [cf(1:k+1); cf(N-k+1:N)];  
>> bdy3 = ifft(cf3);  
>> cf4 = [cf3(1:13); zeros(100,1); cf3(14:25)];  
>> bdy4 = ifft(cf4);
```

MATLAB/Octave

or in Python:

```

In : from numpy.fft import fft, ifft
In : execfile('chaincode.py')
In : z = np.zeros((256,256))
In : z[31:224,31:96] = 1; z[31:96,95:223] = 1; z[192:224,95:160]=1
In : cc, bdy = shape(z)
In : c = bdy[:,0]+bdy[:,1]*1j
In : k = 12; N = len(c)
In : cf = fft(c); cf2 = np.copy(cf); cf2[k+1:N-k]=0; bdy2 = ifft(cf2);
In : cf3 = np.hstack((cf[:k+1],cf[-k:])); bdy3 = ifft(cf3)
In : cf4 = np.hstack((cf[:k+1],np.zeros((100)),cf[-k:])); bdy4 = ifft(cf4)

```

Python

At this stage `cf2` consists of 896 values, most of them zero, and so `bdy2` has 896 values. But `cf3` consists only of the first 13 and last 12 values of `cf`—in effect `cf3` is `cf2` with all the zero elements removed—and so both it and `bdy3` have only 25 values. Finally, `cf4` is created from `cf3` by putting in 100 zeros between the first $k + 1$ and final k values. There is nothing special about 100—any value will do—but the larger the value, the more points will fill out the boundary.

So the plots can be created as

```

>> plot(bdy2, '.'), axis([0,256,0,256]), axis equal, axis nolabel
>> plot([bdy3;bdy3(1)], '-.', "markersize",5), axis([0,10000,0,10000]), ...
    axis equal, axis nolabel
>> plot(bdy4, '.'), axis([0 2000 0 2000]), axis equal, axis nolabel

```

MATLAB/Octave

or as

```

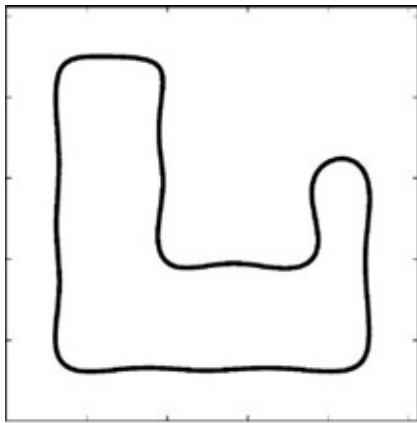
In : plt.plot(bdy2.real, bdy2.imag, '.'); plt.axis('equal')
In : bdy3 = np.hstack(bdy3, bdy3[0])
In : plt.plot(bdy3.real, bdy3.imag, '-.', markersize=5); plt.axis
    ([0,9000,0,9000])
In : plt.plot(bdy4.real, bdy4.imag, '.'); plt.axis([0.0,2000.0,0.0,2000.0])

```

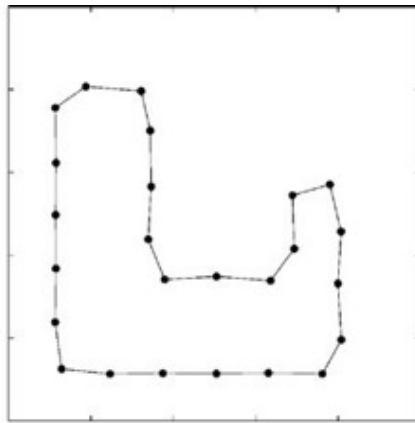
Python

and the results are shown in Figure 12.11.

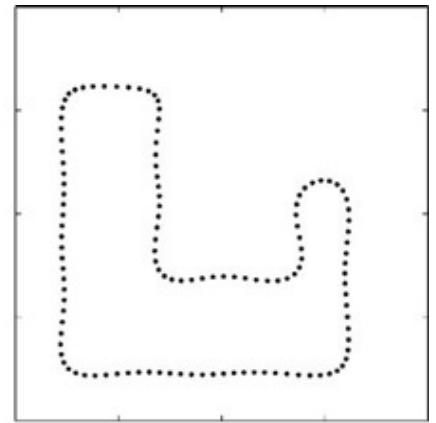
Figure 12.11: Fourier descriptors drawn differently



bdy2: 896 points



bdy3: 25 points joined with lines



bdy4: 25 + 100 points

To give a slightly more substantial example, consider the three shapes in Figure 12.12. Each one can be plotted from a list of points; for example, the plane can be created in MATLAB or Octave with:

```
>> pl = dlmread('plane.txt');  
>> c = complex(pl(:,1),pl(:,2));  
>> plot(c, '.'), axis equal, axis nolabel
```

MATLAB/Octave

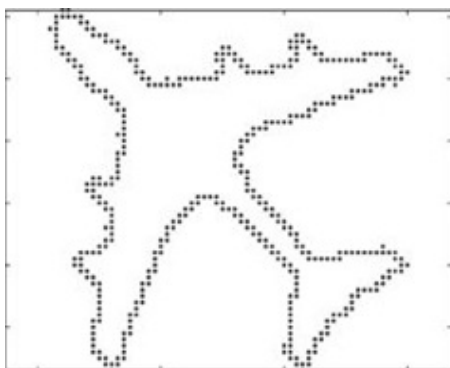
or in Python with

```
In : pl = np.loadtxt('plane.txt')  
In : c = pl[:,0] + pl[:,1]*1j  
In : plt.plot(c.real,c.imag, '.')  
In : plt.axis('equal')  
In : plt.tick_params(axis='both',labelbottom='off',labelleft='off')  
In : plt.show()
```

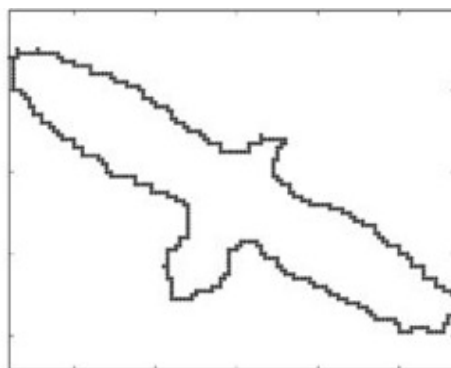
Python

where the `plt.tick_params` function turns off the tick marks on the axes. Each of the three shapes has roughly the same number of boundary points: between 415 and 420.

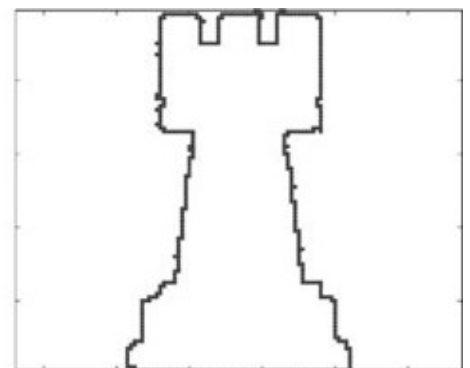
Figure 12.12: Three shapes



plane.txt



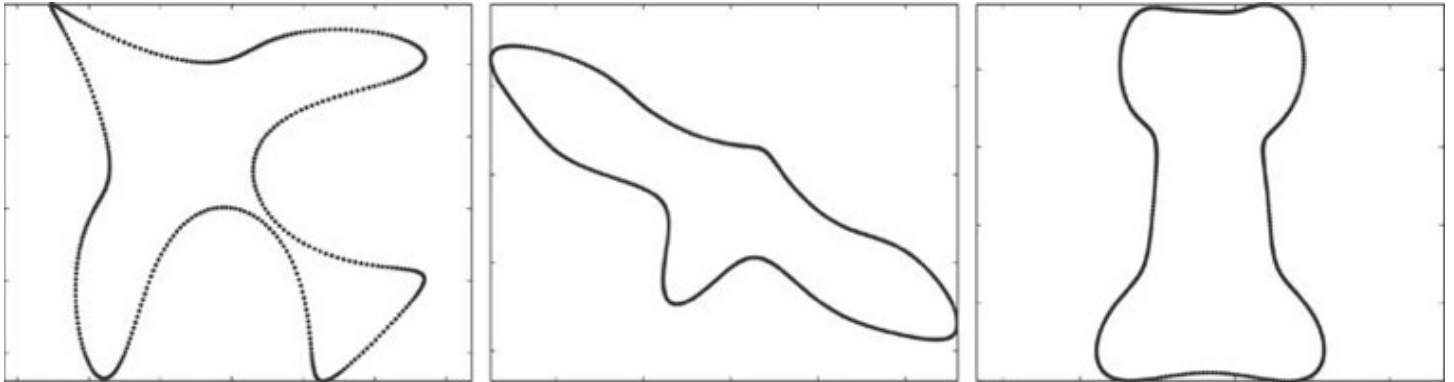
eagle.txt



chess.txt

For each of the values $k = 8, 20, 32$ set all but the first $k + 1$ and last k of the transform values to zero, and invert the result. So Figure 12.13 shows the Fourier descriptors with $k = 8$. Although the boundaries are quite blurred, and fine details are missing, the basic nature of the shapes, and their sizes and orientations are quite clear.

Figure 12.13: Using only 8 pairs of values from the Fourier transform



So Figure 12.14 shows the Fourier descriptors with $k = 20$. With more values used some fine detail begins to appear: the engines on the plane wings and the detail on the top of the chess piece. Even more detail is apparent with $k = 32$, as shown in Figure 12.15.

Although the boundaries are still not as sharp as the originals, a remarkable amount of detail is apparent for only 32 of the roughly 210 pairs of values in the transform.

Figure 12.14: Using 20 pairs of values from the Fourier transform

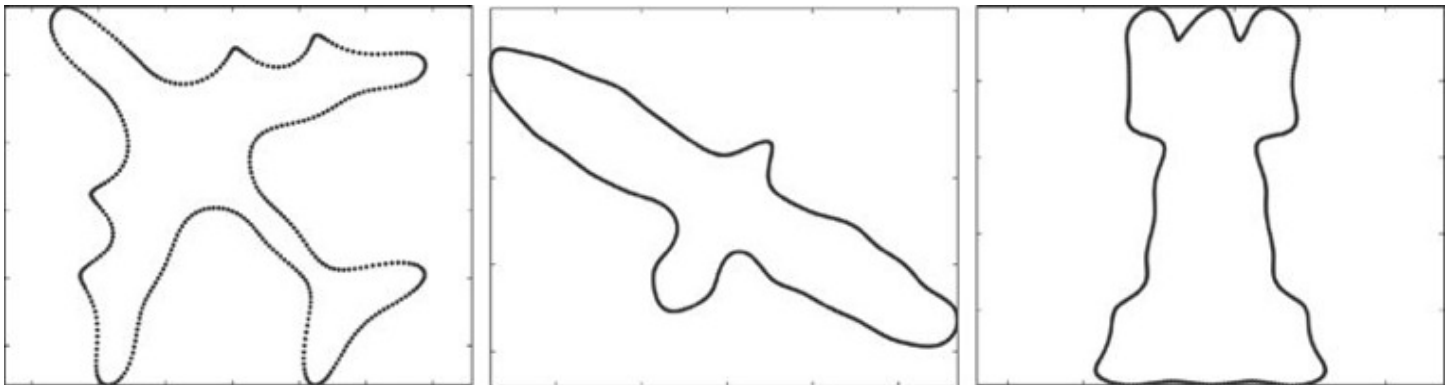
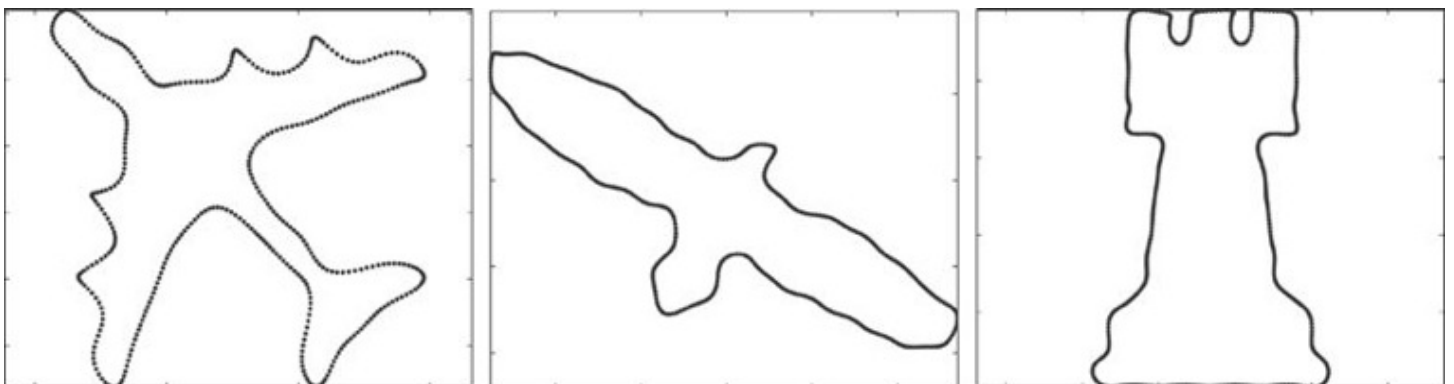


Figure 12.15: Using 32 pairs of values from the Fourier transform



Exercises

1. Find the chain codes, normalized chain codes, and shape numbers for each of the 4-connected shapes given in Figure 12.16.
2. Now repeat the above exercise for all possible reflections and rotations of those shapes.
3. Repeat the above exercises for the 8-connected shapes given in Figure 12.17.
4. Check your answers to the previous questions with the shape program.
5. Generate the shapes with the following 4-connected chain codes:
 - (a) 3 3 3 0 0 0 0 1 1 2 2 1 2 2
 - (b) 3 3 3 0 3 0 0 1 0 1 1 2 2 1 2 2
6. Generate the shapes with the following 8-connected chain codes:
 - (a) 5 6 7 6 0 0 1 2 2 4 3 4

Figure 12.16: Shapes for Question 1

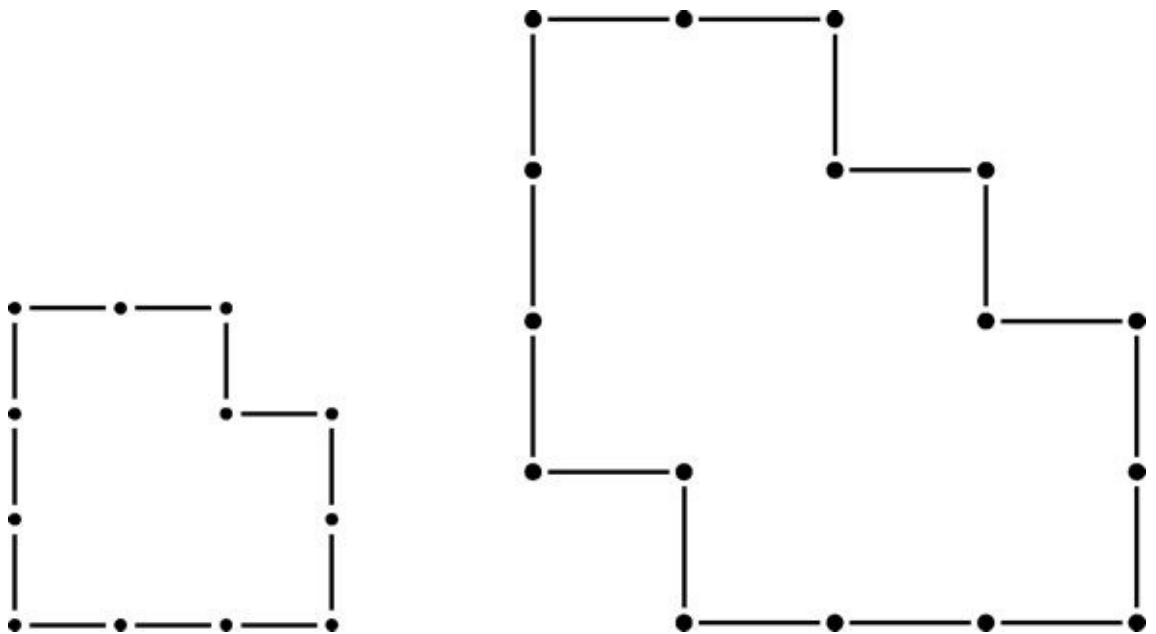
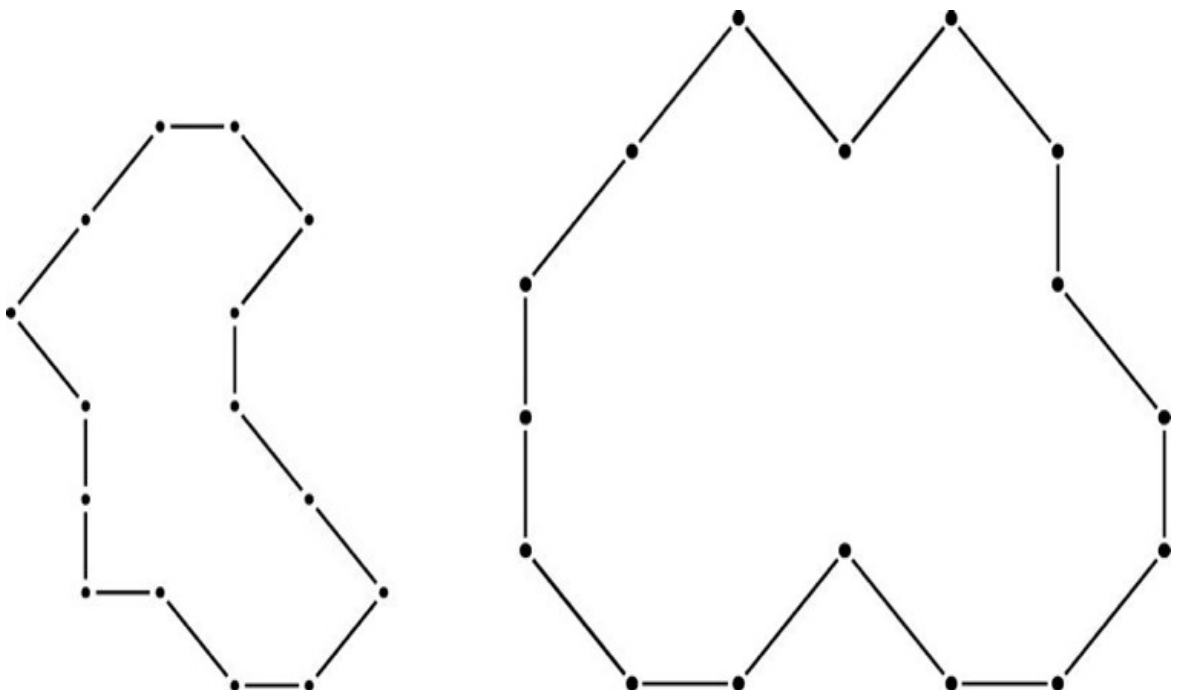


Figure 12.17: Shapes for Question 3



(b) 5 6 7 7 1 7 1 2 2 3 5 3 4

7. Obtain the normalized chaincodes and shape numbers of all the shapes in the previous questions. Check your answers with `shape`.

8. Use your system to generate a T shape, and obtain its Fourier descriptors. How many terms are required to identify the

(a) Symmetry of the object?

(b) Size of the object?

(c) Shape of the object?

9. Repeat the above question but use an X shape. Then again, but with an E shape.

10. Compare the results of the shapes in the previous two questions. How many terms are required to distinguish the

(a) Symmetries

(b) Sizes

(c) Shapes

of the three objects?

11. How does the size of an object affect its Fourier descriptors? Experiment with a 6×4 rectangle, and then with a 12×8 rectangle. Generalize your findings.

13 Color Processing

For human beings, color provides one of the most important descriptors of the world around us. The human visual system is particularly attuned to two things: edges and color. We have mentioned that the human visual system is not particularly good at recognizing subtle changes in gray values. In this section we shall investigate color briefly, and then some methods of processing color images

13.1 What Is Color?

Color study consists of

1. The physical properties of light which give rise to color
2. The nature of the human eye and the ways in which it detects color
3. The nature of the human vision center in the brain, and the ways in which messages from the eye are perceived as color

Physical Aspects of Color

As we have seen in Chapter 1, visible light is part of the electromagnetic spectrum. The values for the wavelengths of blue, green, and red were set in 1931 by the CIE (Commission Internationale d'Eclairage), an organization responsible for color standards.

Perceptual Aspects of Color